High-Level Design (HLD)
RAG-Based Customer Support Assistant

Keerthi Prakash B S

Innomatics Research Labs

April 23, 2026

Contents

1 System Overview **3**
1.1 Problem Definition . . . 3
1.2 Scope of the System . . . 3

2 Architecture Diagram **4**

3 Component Description **5**
3.1 Document Loader . . . 5
3.2 Chunking Strategy . . . 5
3.3 Embedding Model . . . 5
3.4 Vector Store . . . 5
3.5 Retriever . . . 5
3.6 LLM . . . 5
3.7 Graph Workflow Engine . . . 5
3.8 Routing Layer . . . 6
3.9 HITL Module . . . 6

4 Data Flow **6**
4.1 How data moves from PDF → Answer . . . 6
4.1.1 Ingestion Flow (Off-line) . . . 6
4.1.2 Query Flow (On-line) . . . 6
4.2 Query Lifecycle . . . 6

5 Technology Choices **7**
5.1 Why ChromaDB . . . 7
5.2 Why LangGraph . . . 7
5.3 LLM Choice: Groq . . . 7
5.4 Additional Tools . . . 7

6 Scalability Considerations **8**
6.1 Handling Large Documents . . . 8
6.2 Increasing Query Load . . . 8
6.3 Latency Concerns . . . 8

1 System Overview

1.1 Problem Definition

In the modern enterprise landscape, the volume of technical documentation, policy manuals, and product guides is growing at an exponential rate. Customer support departments face a critical "Information Retrieval Bottleneck." Human agents often spend up to 40% of their time searching for specific information within dense PDF documents. This leads to:

- **High Latency**: Customers wait minutes for simple answers.
- **Inconsistency**: Different agents may interpret the same document differently.
- **Cognitive Fatigue**: Agents are overwhelmed by the sheer volume of data they must memorize.

Standard AI chatbots (without RAG) suffer from **Hallucinations**, where they provide confident but factually incorrect answers because they lack access to real-time, proprietary company data.

1.2 Scope of the System

The RAG-Based Customer Support Assistant is designed to be a "Knowledge Navigator." Its primary scope includes:

- **Unstructured Data Ingestion**: Seamlessly converting complex PDFs into searchable knowledge.
- **Semantic Precision**: Understanding the intent behind a query, not just matching keywords.
- **Safe Response Generation**: Only answering queries for which relevant context exists in the documents.
- **Human-in-the-Loop (HITL)**: A robust safety net that escalates complex or sensitive queries to human experts.

The system is intended for use in technical support, HR policy assistance, and internal knowledge management.

2 Architecture Diagram

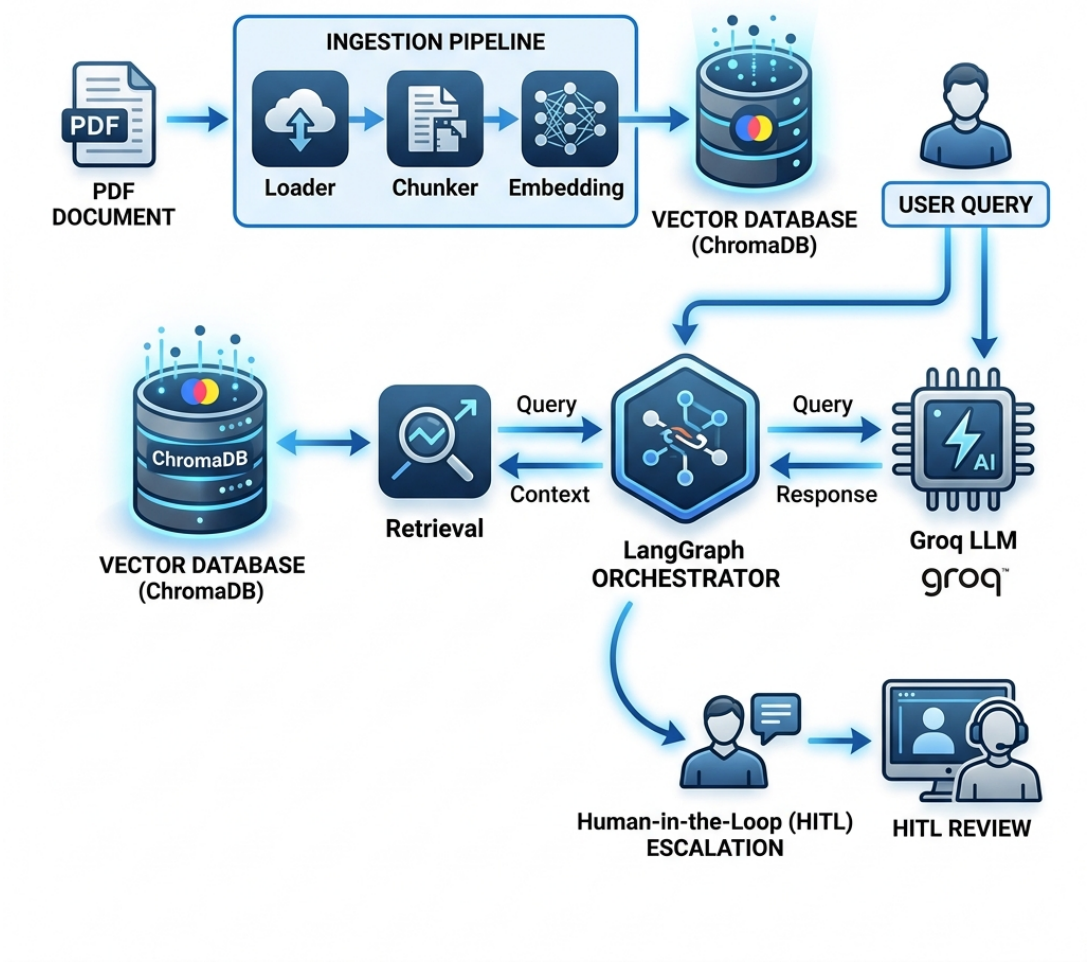

Figure 1: End-to-End System Architecture

The architecture is built on a modular, stateful graph-based design. The components interact as follows:

1. **User Interface**: A unified CLI or Web portal where queries are entered.
2. **Document Ingestion Pipeline**: The "Read" phase that processes raw PDFs.
3. **Embedding System**: The "Translate" phase that converts text into mathematical vectors.
4. **Vector Database (ChromaDB)**: The "Memory" phase that stores these vectors.
5. **Retrieval Layer**: The "Search" phase that finds relevant facts.
6. **LLM Processing Layer (Groq)**: The "Reasoning" phase that synthesizes answers.

7. **Workflow Orchestration (LangGraph)**: The "Brain" that controls the logic and state flow.
8. **HITL System**: The "Safety Gate" that manages human escalation.

3 Component Description

3.1 Document Loader

Utilizes `PyPDFLoader` to perform optical and logical text extraction. It handles multi-column layouts and maintains metadata such as source file name and page numbers.

3.2 Chunking Strategy

We employ a **Recursive Character Splitter**. This strategy is critical for maintaining context. It splits text at logical breaks (paragraphs, then sentences) to keep related information together. A chunk size of 1000 characters with a 200-character overlap is used to bridge context between chunks.

3.3 Embedding Model

The system uses the `all-MiniLM-L6-v2` HuggingFace model. It creates 384-dimensional dense vectors. This model is chosen for its high throughput and low latency on standard hardware.

3.4 Vector Store

ChromaDB serves as our persistent vector database. It uses the HNSW (Hierarchical Navigable Small World) algorithm for efficient nearest-neighbor searches, ensuring sub-100ms retrieval times.

3.5 Retriever

A semantic search component that calculates the Cosine Similarity between the user's query vector and the document vectors. It returns the top 3 most relevant segments.

3.6 LLM

Groq (Llama-3-70b): A state-of-the-art Large Language Model optimized for speed. It generates human-like responses based strictly on the provided context.

3.7 Graph Workflow Engine

LangGraph: Unlike linear pipelines, LangGraph allows for cyclic workflows and stateful node transitions. It manages the "history" and "current status" of every query.

3.8 Routing Layer

A logic gate within the graph that analyzes the LLM's output or user intent to decide whether to finish the interaction or move to the HITL module.

3.9 HITL Module

A dedicated node in the graph that handles handoffs. It formats the query, the attempted context, and the reason for escalation for a human agent's review.

4 Data Flow

4.1 How data moves from PDF → Answer

The data lifecycle is divided into two main processes:

4.1.1 Ingestion Flow (Off-line)

1. **Raw PDF**: Files are uploaded to the `data/` directory. 2. **Extraction**: The Loader extracts text strings. 3. **Normalization**: Cleaning whitespaces and removing artifacts. 4. **Splitting**: Chunking the text into 1000-char blocks. 5. **Vectorization**: Embedding the blocks into numerical arrays. 6. **Indexing**: Storing in ChromaDB for fast retrieval.

4.1.2 Query Flow (On-line)

1. **Input**: User query is received. 2. **Vectorization**: Query is embedded using the same model as ingestion. 3. **Search**: ChromaDB retrieves the top 3 chunks. 4. **Augmentation**: Chunks are combined with the user query into a prompt. 5. **Generation**: LLM processes the prompt. 6. **Validation**: Router checks if the answer is confident or needs escalation.

4.2 Query Lifecycle

The query follows a "State Machine" pattern. It starts in the `START` node, moves to `Process`, and then either goes to `END` or `Human Escalation`.

5 Technology Choices

5.1 Why ChromaDB

We chose ChromaDB because it is **Open Source**, **Locally Persistent**, and requires zero configuration. For enterprise support systems where data privacy is paramount, having a local vector store avoids the risks associated with cloud-based vector databases like Pinecone.

5.2 Why LangGraph

Traditional LangChain chains are "directed acyclic graphs" (DAGs), which make it hard to implement feedback loops or complex human-in-the-loop logic. **LangGraph** treats the workflow as a state machine, allowing us to jump back to previous nodes or pause for human input natively.

5.3 LLM Choice: Groq

Speed is a feature. Groq's LPU architecture delivers Llama-3-70b at speeds exceeding 250 tokens per second. In a support context, a 1-second response time vs. a 10-second response time is the difference between a satisfied customer and a lost one.

5.4 Additional Tools

- **PyPDF**: For robust PDF parsing.
- **Python-Dotenv**: For secure management of API keys.

6 Scalability Considerations

6.1 Handling Large Documents

The system uses incremental ingestion. If a 10,000-page document is added, it is processed in chunks to prevent memory overflow. ChromaDB's persistent storage ensures that we only embed each page once.

6.2 Increasing Query Load

By decoupling the Retriever (CPU/RAM bound) from the LLM (API/GPU bound), we can scale the retrieval layer horizontally. LangGraph instances are lightweight and can handle hundreds of concurrent state transitions.

6.3 Latency Concerns

To minimize latency, we: 1. Use local embeddings (no network trip). 2. Use Groq's high-speed inference. 3. Limit retrieval to top 3 chunks to keep the LLM prompt concise.